

Agriculture Employee Performance Tracking System - System Design

1. System Overview

The system is designed for the Agriculture Department to track operational activities and employee performance using dashboard analytics.

Core objectives:

- Track employee productivity
- Monitor village and farmer assignments
- Manage sugarcane crushing targets
- Track cane registration
- Monitor H&T; agreements
- Analyze recovery operations
- Track KCC and bank loan targets
- Generate dashboards and reports

2. Recommended High-Level Architecture

Users (Admin / Manager / Employee)



React Frontend Dashboard



FastAPI Backend APIs



Business Logic Layer



Analytics Engine



Data Access Layer



CSV / SQLite



Future Microsoft SQL Server Integration

3. Main System Modules

- Authentication Module
- Employee Management Module
- Village Coverage Module
- Farmer Management Module
- Crushing Operations Module
- Cane Registration Module
- Recovery Management Module
- Banking & Loan Tracking Module
- Analytics & Reporting Module
- Dashboard Visualization Module

4. Important Business KPIs

- Village Assigned Count
- Farmer Count
- Crushing Target Achievement
- Cane Registration in Hectares

- H&T; Agreement Completion
- Recovery Pending Amount
- Farmer Meeting Completion
- KCC Loan Achievement
- Canara H&T; Loan Targets
- Sangola H&T; Loan Targets

5. Dashboard Design

Recommended dashboard pages:

1. Executive Dashboard
2. Employee Performance Dashboard
3. Sugarcane Operations Dashboard
4. Recovery Dashboard
5. Farmer Engagement Dashboard
6. Banking & Loan Dashboard

Recommended visualizations:

- KPI Cards
- Bar Charts
- Line Charts
- Pie Charts
- Heatmaps
- Employee Leaderboards

6. Recommended Technology Stack

Frontend:

- React
- Tailwind CSS
- Recharts

Backend:

- FastAPI

Analytics:

- Pandas
- NumPy
- Scikit-learn

Current Storage:

- CSV / SQLite

Future Storage:

- Microsoft SQL Server

7. Future Microsoft SQL Server Integration

Current Flow:

Repository Layer → CSV Reader

Future Flow:

Repository Layer → SQLAlchemy ORM → Microsoft SQL Server

This layered architecture allows easy migration without changing frontend logic.

8. Security Recommendations

- JWT Authentication
- Role-Based Access Control
- Input Validation
- Secure Environment Variables
- API Security
- Password Hashing

9. Development Workflow

Phase 1:

- Dataset Cleaning
- KPI Understanding
- Feature Grouping

Phase 2:

- ER Diagram
- Architecture Diagram
- API Planning

Phase 3:

- Backend Development

Phase 4:

- Frontend Dashboard Development

Phase 5:

- Analytics & Reporting

Phase 6:

- SQL Server Migration

10. Final Recommendations

The project should focus on:

- Clean data organization
- Meaningful business analytics
- Agriculture-specific operational KPIs
- Scalable architecture
- Interactive dashboards
- Future enterprise database integration

The system should be designed using layered architecture so future migration from CSV to Microsoft SQL Server becomes smooth and manageable.